

Ejercicio 18

Dado el siguiente algoritmo exprese, desarrolle e indique el orden de la función de tiempo de ejecución $T(n)$:

```
public static void metodo(int n){
    int i = 1;
    int j = n;
    while (i <= n){
        while (j > 1){
            for (int k = i; k < i+10; k++){
                System.out.println(k*i*j);
            }
            j = j / 2;
        }
        i = i + 2;
    }
}
```

En primer lugar tenemos dos operaciones constantes que unificamos y llamamos c_1 . Luego, tenemos 3 iteraciones.

Analizamos cuantas veces se ejecutan:

- while($i \leq n$)
 1. $i = 1$
 2. $i = 1 + 2$
 3. $i = 1 + 2 + 2 = 1 + 2 \cdot 2$
 4. $i = 1 + 2 \cdot 2 + 2 = 1 + 2 \cdot 3$
 - $\vdots$

Paso h : $i = 1 + 2(h - 1)$

Entonces, cuando :

$$\begin{aligned}i &= n \\n &= 1 + 2(h - 1) \\n - 1 &= 2h - 2 \\ \frac{n + 1}{2} &= h\end{aligned}$$

La primera iteración se ejecuta $\frac{n+1}{2}$ veces.

- while(j<1) Como j no se inicializa en cada iteración del primer while, sólo se va a ejecutar cuando $i = 1$. Veamos los valores de j .

1. $j = 1$
2. $j = \frac{n}{2}$
3. $j = \frac{n}{2 \cdot 2} = \frac{n}{2^2}$
4. $j = \frac{n}{2^2 \cdot 2} = \frac{n}{2^3}$
- $\vdots$

Paso f : $j = \frac{n}{2^{f-1}}$
 Cuando

$$\begin{aligned} j &= 1 \\ 1 &= \frac{n}{2^{f-1}} \\ 2^{f-1} &= n \\ f &= 1 + \log_2 n \end{aligned}$$

Cuando $j = 1$, $f = 1 + \log_2 n$ pero como la condición de corte es $j > 1$, la iteración se ejecuta $\log_2 n$ veces.

- for (int $k = i$; $k \leq i+10$; $k++$) Como k aumenta de a 1, esta iteración se ejecuta 10 veces en cada iteración de j .

Entonces tenemos

- $c_1 \rightarrow$ Operaciones constantes del inicio
- $\sum_{k=1}^{10} c_2 \rightarrow$ Ejecuciones del for (tercera iteración)
- $\sum_{j=1}^{\log_2 n} [\sum_{k=1}^{10} c_2] \rightarrow$ Ejecuciones del while interno (segunda iteración). Este término corresponde a la primera iteración del primer while, cuando $i = 1$.
- $\sum_{i=2}^{\frac{n+1}{2}} c_3 \rightarrow$ Ejecuciones restantes del primer while. En estos casos no se ejecutan las iteraciones internas porque $j = 1$, no entra en el segundo while.

Tenemos

$$\begin{aligned}
T(n) &= c_1 + \sum_{j=1}^{\log_2 n} \left[\sum_{k=1}^{10} c_2 \right] + \sum_{i=2}^{\frac{n+1}{2}} c_3 \\
&= c_1 + \sum_{j=1}^{\log_2 n} c_2(10 + 1 - 1) + c_3 \sum_{i=2}^{\frac{n+1}{2}} 1 \\
&= c_1 + 10c_2 \sum_{j=1}^{\log_2 n} 1 + c_3 \left(\frac{n+1}{2} - 2 + 1 \right) \\
&= c_1 + 10c_2(\log_2 n + 1 - 1) + c_3 \left(\frac{n+1}{2} \right) \\
&= c_1 + \frac{c_3}{2} + 10c_2 \log_2 n + \frac{c_3}{2} \\
\text{Renombramos } c_4 &= c_1 + \frac{c_3}{2}, c_5 = 10c_2 \text{ y } c_6 = \frac{c_3}{2} \\
&= c_4 + c_5 \log_2 n + c_6 n
\end{aligned}$$

Entonces, la función de tiempo de ejecución del método es

$$T(n) = c_4 + c_5 \log_2 n + c_6 n$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = c_4 + c_5 \log_2 n + c_6 n \leq O(n)$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_4 + c_5 \log_2 n + c_6 n \leq c \cdot n, \text{ para todo } n \geq n_0$$

Se tiene que demostrar que

$$T(n) = c_4 + c_5 \log_2 n + c_6 n \leq O(n)$$

Para esto, tenemos que encontrar una constante $c > 0$ y un n_0 tales que

$$c_4 + c_5 \log_2 n + c_6 n \leq cn \text{ para todo } n \geq n_0$$

Analizamos término a término,

- Primer término, c_4
Sabemos que $1 \leq n$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \leq c_4 n$$

Encontramos un valor $k_1 = c_4$. Ahora, si probamos con $n_0 = 1$.

$$c_4 \leq c_4 \cdot 1$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_4$ y $n_0 = 1$, para $n_0 \geq 1$.

- Segundo término, $c_5 \log_2 n$
Sabemos que $\log_2 n \leq n$. Si multiplicamos a ambos lados por c_5 .

$$c_5 \log_2 n \leq c_5 n$$

Encontramos un valor $k_2 = c_5$. Ahora, si probamos con $n_0 = 1$.

$$c_5 \log_2 1 \leq c_5 \cdot 1$$

$$c_5 \cdot 0 \leq c_5$$

$$0 \leq c_5$$

La desigualdad se cumple porque $c_5 \geq 0$. Los valores que acotan al término son $k_2 = c_5$ y $n_0 = 1$.

- Tercer término, $c_6 n$ Sabemos que $n \leq n$. Si multiplicamos a ambos lados por c_6 .

$$c_6 n \leq c_6 n$$

Encontramos un valor $k_3 = c_6$. Ahora, si probamos con $n_0 = 0$.

$$c_6 \cdot 0 \leq c_6 \cdot 0$$

$$0 \leq 0$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_6$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_4 + c_5 \log_2 n + c_6 n \leq k_1 n + k_2 n + k_3 n$$

$$T(n) \leq (k_1 + k_2 + k_3)n$$

$$T(n) \leq (c_4 + c_5 + c_6)n$$

$$\text{Renombramos } c = c_4 + c_5 + c_6$$

$$T(n) \leq cn$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_4 + c_5 + c_6 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$